HLD focuses on Architecture of Application

Software/Application includes

↳ websites
↳ Mobile Application
↳ Software

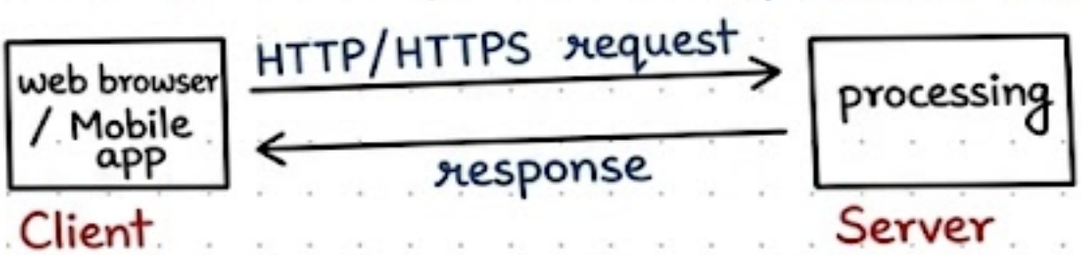
→ focus of HLD (related to internet)

↳ HLD do not focus much on s/w as suppose Adobe photoshop s/w software it uses your computer resource like ram & memory and it depends on user whether to update to its newer version or not & there is no multiple user problem.

↳ this kind of s/w has more focus on LLD rather than HLD.

↳ Just for info: How you get Response?

↳ Request Response Model / Client - Server Model



- Client could be any frontend as - Mobile app (android / ios) and web-browser jiske through aap request bhej skte ho using HTTP/HTTPS request ke through to server, and after processing server respond karta hai & that's called request-response model.

- this will work fine till limited users access our application

  Lekin ek time pe jab no. of users (millions of user approx) ek sath request karte hai server pe toh server crash ho jaata hai due to unable to serve all request as count of request are much more than it is able to serve.

- How to handle it → Distributed system

Distributed system ?

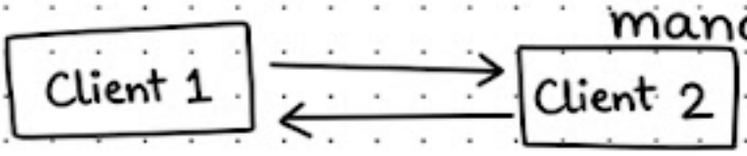
refers to a system where components (Server, DB or Application) are spread across multiple networks.

- Horizontal scaling Vs Vertical scaling

  different methods to handle multiple user by adding or increasing capacity of servers.

5) Websocket - peer to peer communication

b no server involved to manage communication



Difference b/w websocket and webRTC

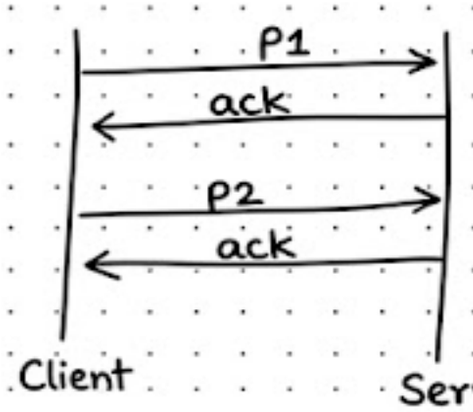
websocket → Client directly interacts ⇒ no server in b/w

webRTC → Communication through server.

- **Transport Layer → TCP , UDP**

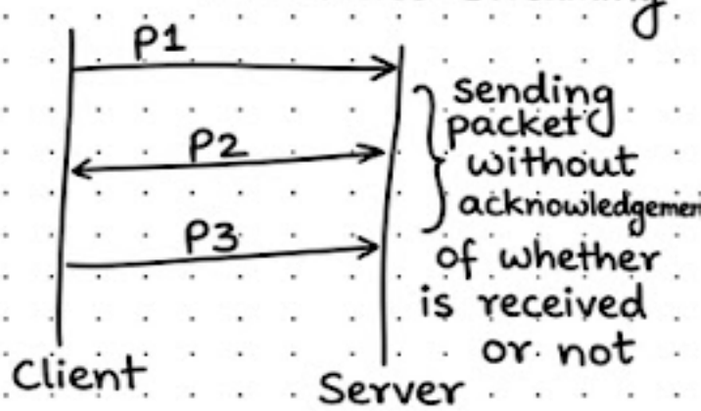
↳ TCP

b Transmission control Protocol
b Reliable
b Slow transmission



↳ UDP

b User Datagram Protocol
b Non-Reliable
b fast transmission
ext: Video Streaming



* A message is divided into Small packets

DNS

Clients do not directly communicate with Server

↳ Client first communicate with DNS

↳ DNS returns server IP Address

↳ Then client requests to that IP Address.

Why DNS ?

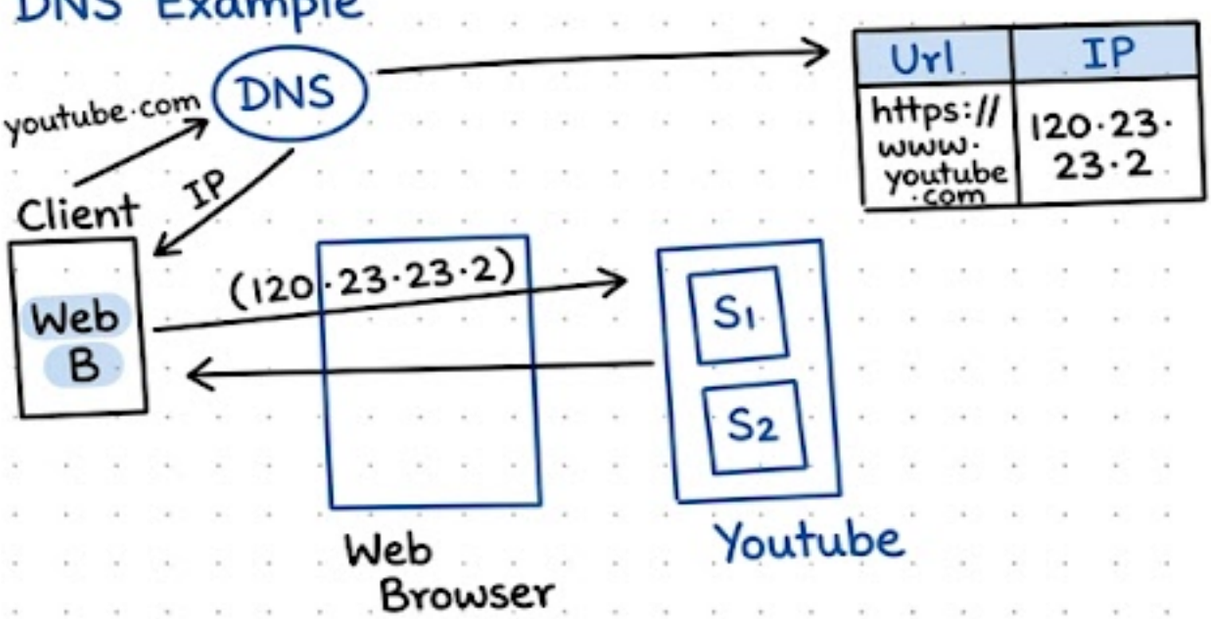
↳ Client asks as domain name of that application but we need IP address

↳ address of application where they are hosted)

↳ DNS stores its domain-name : IP-address

↳ When client asks it returns IP address

DNS Example



Lecture 1: Introduction to HLD

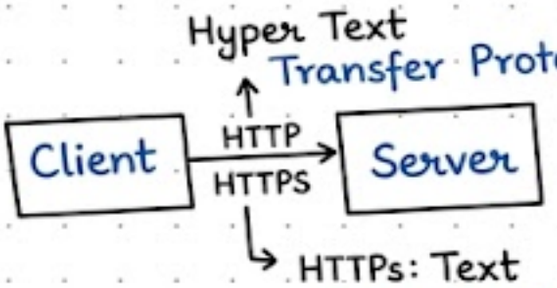
System design - Divided into two parts

① HLD (Focuses on))
- main focus is on architecture
- crucial phase as it decides what to choose for developing application based on trade-off and requirements

- Architecture
- Tech Stack → MERN, SPRINGBOOT, DJANGO
- Cost optimization
- Database → SQL, NOSQL, GraphQL
- USER → million of user should handle concurrently
  - we also have to estimate no. of users that can come so that they are handled ⇒ How do we estimate? ⇒ Using envelope estimate

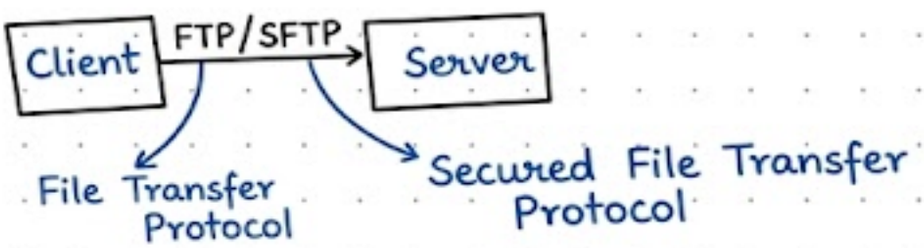
② LLD (focuses on)
- coding part
  - OOPs
  - Design principle (SOLID)
- Clean Code
- Loosely Coupled Code

1> HTTP / HTTPS : request over internet
↳ load webpage
↳ open application



↳ HTTPs: Text Transfer Protocol Secure } used when authentication / Secure needed things are required

2> FTP/SFTP → when we are sharing files/pdfs over Internet



3> SMTP
↳ Simple Mail Transfer protocol
↳ used when we are sending mails over Internet

POP3/IMAP → used to receive mail over Internet

4> WebRTC → used in application where server can share response without client requesting it

e.g. Whats App
↳ Client 1 wants to send messages to client 2 (can be done through HTTP)
↳ Server should deliver that message to client 2 without client 2 requesting for it.

Network Communication

Network Communication are rules that both Client and Server follow to communicate over internet.

Applications follows any of these 2 models

ISO-OSI model (7-layers)

- Each layer has it's own protocol

TCP/IP (5-layers)

- Each layer has its own protocol

- **Network Protocols (Rules)**

Let's discuss of OSI Model (7 layers)

↳ Application Layer
↳ Presentation Layer
↳ Session Layer
↳ Transport Layer
↳ Network Layer
↳ DataLink Layer
↳ Physical Layer

we will discuss only these layer's protocol not other things as they don't have role in HLD

- Application → HTTP, HTTPS, FTP, SFTP, SMTP, WeBRTC, WebSocket

Client when communicate with server these rules will be followed depend on request of Client.

Vertical Scaling

→ Increasing capacity of already existing server like RAM = 32GB → 64GB

disadvantages

↳ there is limitation to maximum increase of RAM as CPU should be able to handle it

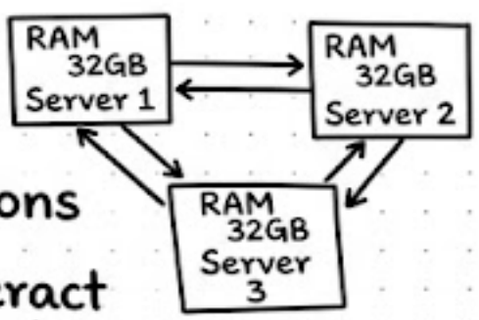
↳ No increase in computational power

↳ Good for small applications

Horizontal Scaling

→ Increasing no. of servers to handle multiple users

→ Good for large applications



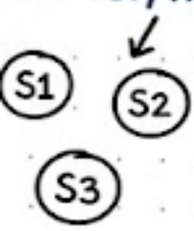
But how will server interact with each other? How data will be stored in all servers with consistency? How we will decide who will handle which server user

↳ answer in coming lectures

- Vertical scaling & Horizontal scaling is base to understand distributed systems.

- In HLD → Server are known as nodes (So, S1, S2, S3 will be discussed later)

Server/nodes



- But this creates complexity in our website as server will interact with each other to maintain data integrity
- Distributed system → complex architecture

[But solves huge problems]